

Học sâu cho phân đoạn ảnh

UNet, hàm mất mát, độ đo, dữ liệu

Thanh-Sach LE

✉ Itsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology
Ho Chi Minh City. Ngày 10 tháng 4 năm 2026

Mục lục

- 1 Giới thiệu và định nghĩa bài toán
- 2 Kiến trúc lõi: encoder-decoder và UNet
- 3 Hàm mất mát
- 4 Độ đo và đánh giá
- 5 Data pipeline và loader
- 6 Chủ đề nâng cao



Image Segmentation

1 Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Giới thiệu và định nghĩa bài toán

Image segmentation là gì?

- Gán **nhãn lớp** cho **từng pixel** trong ảnh (dense prediction).
- Đầu ra là một **label map** được căn chỉnh theo lưới pixel của ảnh.

Góc nhìn tensor

Batch đầu vào $\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W}$. Logits $\mathbf{Z} \in \mathbb{R}^{B \times K \times H \times W}$ cho K lớp.

Image Segmentation

Mục lục

3 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Classification vs detection vs segmentation



Image Segmentation

Mục lục

4 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Task	Output	Structure	Typical metric
Classification	1 label / image	Low	Accuracy
Detection	Boxes + classes	Medium	mAP
Segmentation	Mask / masks	High	mIoU, Dice, PQ

Takeaway

Segmentation kết hợp **đặc trưng thị giác** với **bố cục không gian**.

Semantic segmentation



Image Segmentation

Mục lục

5 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Mỗi pixel nhận một nhãn lớp; **không phân biệt từng đối tượng riêng lẻ.**
- Đầu ra: class index map $\mathbf{Y} \in \{1, \dots, K\}^{H \times W}$ cho mỗi ảnh.

Ứng dụng điển hình

Scene parsing, phân vùng mô/tạng trong y khoa, bản đồ lớp phủ đất.

Instance segmentation



Image Segmentation

Mục lục

6 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Tách riêng **các đối tượng** thuộc cùng một lớp: “đối tượng nào là đối tượng nào”.
- Đầu ra thường là một **tập** các mask, kèm class id và score.

Khó hơn semantic segmentation

Cần xử lý grouping và duplicate suppression; các lỗi kiểu detection cũng xuất hiện.

Panoptic segmentation



Image Segmentation

Mục lục

7 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Kết hợp **things** (đối tượng đếm được) và **stuff** (vùng nền/không có cá thể rõ).
- Các vùng không chồng lấn và phủ **toàn bộ pixel**.

Trực giác về metric

PQ (Panoptic Quality) được tách thành $\mathbf{SQ} \times \mathbf{RQ}$.

Ý nghĩa ngắn gọn

SQ đo *mask overlap*; RQ đo *phát hiện/ghép cặp*; PQ cần cả hai tốt.

Panoptic segmentation: SQ, RQ, PQ



Image Segmentation

Mục lục

8 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- **SQ** (Segmentation Quality): độ khớp mask trung bình trên các cặp match đúng,

$$SQ = \frac{\sum_{(p,g) \in TP} \text{IoU}(p, g)}{|TP|}.$$

- **RQ** (Recognition Quality): chất lượng nhận dạng/ghép cặp instance,

$$RQ = \frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}.$$

- **PQ**:

$$PQ = SQ \times RQ.$$

Vì sao segmentation quan trọng?



Image Segmentation

Mục lục

9 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- **Medical:** biên của cơ quan/khối u ảnh hưởng trực tiếp đến quyết định lâm sàng.
- **Driving / robotics:** vùng di chuyển được, lane, vật cản.
- **Satellite / industry:** thửa đất, lỗi bề mặt, vùng vật liệu.

Góc nhìn kỹ thuật

Lỗi thường nằm ở **boundary** và **small object**, không chỉ là “sai lớp”.

Logits và xác suất



Image Segmentation

Mục lục

10 Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Logits theo từng pixel $z_{i,k}$; softmax cho ta xác suất lớp

$$p_{i,k} = \frac{\exp(z_{i,k})}{\sum_{c=1}^K \exp(z_{i,c})}.$$

- Khi huấn luyện, loss thường được tính trên logits hoặc xác suất.
- Khi triển khai, thường dùng $\arg \max_k p_{i,k}$ để tạo class map.

Kiến trúc lõi: encoder-decoder và UNet

Dense prediction pipeline — level 0 (overview)



Image Segmentation

Mục lục

Giới thiệu

12 Kiến trúc Unet

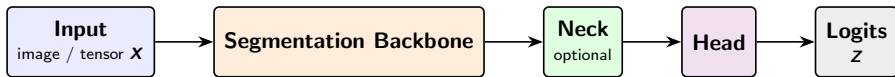
Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết



$$X \rightarrow \text{Backbone} \rightarrow \text{Neck(optional)} \rightarrow \text{Head} \rightarrow Z, \quad Z \in \mathbb{R}^{B \times K \times H \times W}.$$

- **Backbone:** bộ máy trích đặc trưng và tái dựng cấu trúc không gian.
- **Neck:** tầng tùy chọn để refinement/fusion/adaptation trước head.
- **Head:** phép chiếu sang đầu ra của bài toán, thường là Conv2d 1×1 với K filters.

Dense prediction pipeline — level 2 (UNet-style backbone)



Image Segmentation

Mục lục

Giới thiệu

13 Kiến trúc Unet

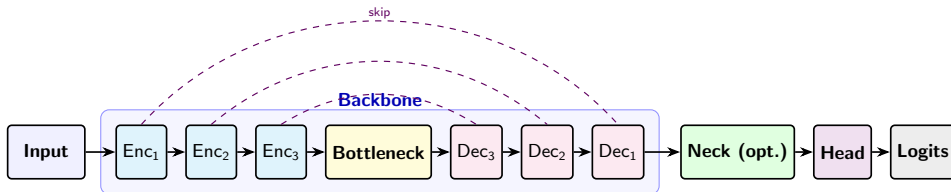
Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết



- **Backbone** bao gồm **encoder + bottleneck + decoder + skip links**.
- **Skip connections** đưa các feature map độ phân giải cao từ encoder sang decoder để giữ biên sắc nét.
- Với UNet cổ điển, neck là identity/không có; trong các mô hình hiện đại, neck có thể là Conv refinement, ASPP, attention, fusion, hoặc boundary refinement.

Classification backbone vs segmentation backbone



- Classification backbone thường chỉ là encoder, sau đó có global pooling.
- Segmentation backbone phải giữ hoặc tái dựng cấu trúc không gian.

Task family	Typical pipeline
Classification	image $\rightarrow$ encoder $\rightarrow$ pooled vector $\rightarrow$ FC
Segmentation	image $\rightarrow$ encoder $\rightarrow$ bottleneck $\rightarrow$ decoder $\rightarrow$ dense map

Takeaway

Ý nghĩa của “backbone” phụ thuộc vào loại bài toán.

Image Segmentation

Mục lục

Giới thiệu

14 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

UNet như một segmentation backbone tổng quát



Image Segmentation

Mục lục

Giới thiệu

15 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

$\text{Backbone}_{\text{UNet}} = \text{Encoder} + \text{Bottleneck} + \text{Decoder} + \text{SkipConnections}.$

$\text{Backbone}_{\text{UNet}} = \text{ClassificationBackbone} + \text{Bridge/Bottleneck} + \text{Decoder}.$

- Encoder có thể là conv blocks tự thiết kế hoặc backbone tiền huấn luyện như ResNet/EfficientNet/ConvNeXt/ViT/Swin.
- Decoder chuyển đặc trưng encoder thành đặc trưng phù hợp cho dense prediction.
- Skip connections khôi phục thông tin định vị và chi tiết biên.

Neck như tầng tinh chế đặc trưng trước head

$$\mathbf{F}_{\text{neck}} = g_{\phi}(\mathbf{F}_{\text{backbone}}), \quad \mathbf{Z} = \text{Conv}_{1 \times 1}(\mathbf{F}_{\text{neck}}).$$

- Neck là tầng tùy chọn nằm giữa backbone và head.
- Vai trò: refine, fuse, adapt, hoặc enrich các dense feature maps.

Neck type	Example
Identity	classical UNet [1]
Conv neck	one/more Conv2d blocks
ASPP neck	DeepLab/ASPP context [2]
Attention neck	SE [3] / CBAM [4] / Transformer attention [5]
Fusion neck	FPN [6] / BiFPN [7] / SegFormer MLP fusion [8]
Boundary neck	edge refinement module

Head không phải là decoder

- Decoder tái dựng feature maps theo không gian.
- Neck tinh chế feature maps sau decoder nếu có.
- Head chiếu số kênh đặc trưng sang đầu ra của bài toán.

$$\mathbf{F} \in \mathbb{R}^{B \times C \times H \times W}, \quad \text{Head}(\mathbf{F}) = \text{Conv2d}(C, K, 1)(\mathbf{F}), \quad \mathbf{Z} \in \mathbb{R}^{B \times K \times H \times W}.$$

Takeaway

Head thường nhỏ, nhưng nó quyết định ngữ nghĩa của đầu ra.

Image Segmentation

Mục lục

Giới thiệu

17 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Ánh xạ các mô hình phổ biến vào taxonomy



Model	Backbone	Neck	Head
Classical UNet [1]	Enc-Bot-Dec + skips	Identity / none	1×1 Conv
ResUNet [11]	ResNet encoder + UNet decoder	optional Conv	1×1 Conv
DeepLabv3+ [2]	CNN encoder + decoder	ASPP + refinement	1×1 Conv
SegFormer [8]	Hierarchical Transformer encoder	MLP fusion neck	linear / 1×1
Mask2Former [9]	Pixel encoder + Transformer decoder	query/mask refinement	mask-class heads
SAM-like [10]	Image encoder + prompt/mask decoder	prompt-conditioned refinement	mask prediction head

Image Segmentation

Mục lục

Giới thiệu

18 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Cách đọc một paper segmentation bất kỳ

- Encoder / visual backbone là gì?
- Mô hình khôi phục độ phân giải không gian bằng cách nào?
- Có skip connection hoặc multi-scale fusion nào?
- Có neck không, và neck thực hiện refinement kiểu gì?
- Head có định dạng đầu ra như thế nào?
- Loss và metric nào định nghĩa thành công?

Takeaway

Phần lớn paper thay đổi backbone, neck, head, loss, hoặc training protocol.

Image Segmentation

Mục lục

Giới thiệu

19 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Mental model khuyến nghị trong học phần này



Basic UNet

Input $\rightarrow$ UNet Backbone $\rightarrow$ Head

Practical modern segmentation

Input $\rightarrow$ Visual Encoder $\rightarrow$ Decoder $\rightarrow$ Neck(optional) $\rightarrow$ Head

Research view

Input $\rightarrow$ Backbone $\rightarrow$ Neck(s) $\rightarrow$ Task Head(s) $\rightarrow$ Losses/Metrics

Quan trọng

Taxonomy này không nhằm tranh luận tên gọi; nó nhằm phân tích chức năng.

Image Segmentation

Mục lục

Giới thiệu

20 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Vì sao CNN phân loại thuần túy là chưa đủ?

- Global pooling làm mất thông tin **ở đâu**.
- Segmentation cần **quyết định theo từng pixel** với tín hiệu định vị.

Takeaway

Cần một pathway có khả năng giữ lại hoặc tái dựng **chi tiết không gian**.

Image Segmentation

Mục lục

Giới thiệu

21 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Encoder: conv blocks

- Gồm các convolution xếp chồng, nonlinearity và normalization.
- Học texture cục bộ và dần dần học các mẫu **ngữ nghĩa** hơn.



Image Segmentation

Mục lục

Giới thiệu

22 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Encoder: downsampling



Image Segmentation

Mục lục

Giới thiệu

23 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Pooling hoặc strided conv làm giảm (H, W) và mở rộng receptive field.
- Tạo ra một **feature hierarchy**: từ chi tiết mịn $\rightarrow$ biểu diễn thô hơn.

Bottleneck



Image Segmentation

Mục lục

Giới thiệu

24 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Feature map sâu nhất, thô nhất, chứa **context** mạnh nhất.
- Thường có nhiều kênh nhất; chi phí tính toán trên mỗi pixel cao nhưng số pixel ít hơn.

Decoder: upsampling



Image Segmentation

Mục lục

Giới thiệu

25 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Tăng dần độ phân giải về gần kích thước đầu vào bằng transpose conv, interpolate+conv, v.v.
- Nếu không có skip connections, upsampling đơn thuần thường tạo mask mờ.

Decoder: feature fusion



Image Segmentation

Mục lục

Giới thiệu

26 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Kết hợp thông tin ngữ nghĩa thô với tín hiệu không gian mịn hơn.
- UNet dùng **skip connections** từ encoder sang decoder.

Skip connections: vì sao cần?

- Pooling làm mất các **biên và cấu trúc mảnh** có tần số cao.
- Skip truyền trực tiếp thông tin định vị sang decoder.

Insight

Skip là cơ chế chính giúp CNN U-Net tạo biên sắc nét.



Image Segmentation

Mục lục

Giới thiệu

27 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Concatenation vs addition



Image Segmentation

Mục lục

Giới thiệu

28 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- **Concat**: giữ đặc trưng encoder và decoder tách biệt, phổ biến trong UNet.
- **Add**: cần số kênh khớp nhau; rẻ hơn nhưng có thể làm mờ vai trò của hai nhánh.

Thực hành

Concat thường an toàn hơn khi chi tiết biên là quan trọng.

Vì sao UNet hiệu quả? Góc nhìn kỹ thuật



Image Segmentation

Mục lục

Giới thiệu

29 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Tách rõ: **context** ở tầng sâu + **localization** qua skip.
- Có inductive bias mạnh cho đầu ra dense trên lưới ảnh.

Memory footprint



Image Segmentation

Mục lục

Giới thiệu

30 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- VRAM tăng theo batch, độ phân giải, số kênh skip và độ rộng decoder.
- Patch training, mixed precision, và giảm base channels có thể giúp tiết kiệm bộ nhớ.

Takeaway

Segmentation thường chạm giới hạn bộ nhớ trước khi chạm giới hạn độ chính xác.

Segmentation head



Image Segmentation

Mục lục

Giới thiệu

31 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Được áp dụng sau decoder output, hoặc sau neck output nếu có neck.
- Thường là convolution 1×1 : chuyển số kênh $\rightarrow K$ logits cho mỗi pixel.

Shape

Feature map (B, C, H, W) trở thành logits (B, K, H, W) .

Outputs: softmax, sigmoid, argmax



Image Segmentation

Mục lục

Giới thiệu

32 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- **Multi-class** ($K > 2$): dùng softmax theo chiều lớp.
- **Binary**: đôi khi dùng sigmoid theo từng pixel hoặc từng channel.
- **Hard labels**: $\hat{y}_i = \arg \max_k p_{i,k}$.

Binary vs multiclass



Image Segmentation

Mục lục

Giới thiệu

33 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- **Multiclass softmax**: mỗi pixel thuộc đúng một lớp loại trừ lẫn nhau.
- **Binary**: foreground/background hoặc các channel độc lập trong thiết lập multi-label.

Ghép loss đúng

CE hay BCE phụ thuộc vào cách biểu diễn nhãn: class index hay binary masks theo từng lớp.

Tensor shapes trong một UNet ví dụ



Image Segmentation

Mục lục

Giới thiệu

34 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Stage	Typical map shape idea
Input	$(B, 3, H, W)$
Encoder levels	$(B, c_\ell, H_\ell, W_\ell)$ với H_ℓ giảm dần
Bottleneck	(B, c_L, H_L, W_L)
Decoder levels	upsample dần về $(B, \cdot, H, W)$
Logits	(B, K, H, W)

Sai khác kích thước: logits vs target

- Logits của mô hình có thể là (B, K, H', W') , trong khi target mask là (B, H, W) .
- Với CE loss, kích thước không gian của logits và target phải khớp.

```
logits = model(images) # (B, K, H', W')
target = masks_long # (B, H, W)
logits = F.interpolate(
    logits, size=target.shape[-2:],
    mode="bilinear", align_corners=False
)
loss = ce(logits, target)
```

Cảnh báo

Không bao giờ bilinear-resize class-index masks; với mask phải dùng nearest-neighbor.

Image Segmentation

Mục lục

Giới thiệu

35 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Các biến thể thực hành



Image Segmentation

Mục lục

Giới thiệu

36 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

UNet++	Nested skip pathways
Attention UNet	Gates skip contribution
Residual UNet	Residual blocks in encoder/decoder

- Các biến thể có thể thay encoder, decoder, neck, hoặc skip fusion nhưng vẫn giữ logic pipeline chung.

CNN vs Transformer segmentation: trực giác



Image Segmentation

Mục lục

Giới thiệu

37 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- **CNN UNet**: có local prior mạnh; hiệu quả ở độ phân giải vừa phải.
- **Transformer**: trộn thông tin xa tốt hơn; thường tốn bộ nhớ hơn ở full resolution.

Hybrid

Nhiều hệ thống dùng transformer encoder + CNN/UNet decoder để cân bằng hai phía.

Code: DoubleConv



Image Segmentation

Mục lục

Giới thiệu

38 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

```
class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.net(x)
```

Ghi chú

Hai conv mỗi stage là conv block tối thiểu rất phổ biến.

Code: encoder step (Down)

```
class Down(nn.Module):  
    def __init__(self, in_ch, out_ch):  
        super().__init__()  
        self.conv = DoubleConv(in_ch, out_ch)  
        self.pool = nn.MaxPool2d(2)  
    def forward(self, x):  
        skip = self.conv(x)  
        return self.pool(skip), skip
```

Ghi chú

Trả về cả tensor đã pool cho nhánh sâu hơn và skip cho decoder concat.



Image Segmentation

Mục lục

Giới thiệu

39 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Code: decoder step (Up)



```
import torch
import torch.nn as nn

class Up(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.up = nn.ConvTranspose2d(in_ch, in_ch // 2, 2, stride=2)
        self.conv = DoubleConv(in_ch, out_ch)
    def forward(self, x, skip):
        x = self.up(x)
        if skip.shape[-2:] != x.shape[-2:]:
            skip = skip[..., : x.size(-2), : x.size(-1)]
        x = torch.cat([x, skip], dim=1)
        return self.conv(x)
```

Image Segmentation

Mục lục

Giới thiệu

40 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Ghi chú

Crop hoặc resize skip để khớp với tensor đã upsample trước khi concat.

Code: segmentation head (SegHead)

```
class SegHead(nn.Module):  
    def __init__(self, in_ch, n_classes):  
        super().__init__()  
        self.proj = nn.Conv2d(in_ch, n_classes, kernel_size=1)  
    def forward(self, x):  
        return self.proj(x)
```

Ghi chú

Không dùng softmax ở đây nếu huấn luyện bằng CrossEntropyLoss.

Image Segmentation

Mục lục

Giới thiệu

41 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Code: UNet.__init__ (conceptual sketch)

```
class UNet(nn.Module):
    def __init__(self, n_classes, in_ch=3, c=(64,128,256,512)):
        super().__init__()
        self.downs = nn.ModuleList()
        prev = in_ch
        for ch in c:
            self.downs.append(Down(prev, ch)); prev = ch
        self.bot = DoubleConv(prev, prev * 2)
        self.ups = nn.ModuleList()
        rev = list(reversed(c))
        ...
        self.head = SegHead(rev[0], n_classes)
```

Ghi chú

Đây là sketch để hiểu cách nối module, không phải implementation hoàn chỉnh để copy-paste chạy ngay.

Image Segmentation

Mục lục

Giới thiệu

42 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Code: UNet.forward (pattern)



Image Segmentation

Mục lục

Giới thiệu

43 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

```
def forward(self, x):
    skips = []
    for down in self.downs:
        x, s = down(x); skips.append(s)
    x = self.bot(x)
    for up, sk in zip(self.ups, reversed(skips)):
        x = up(x, sk)
    return self.head(x)
```

Ghi chú

Đầu ra là logits có shape (B, K, H, W) .

Code: training step cho một batch

```
logits = model(images)
loss = criterion(logits, masks_long)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

Contract

masks_long có shape (B, H, W) và chứa class indices.

Image Segmentation

Mục lục

Giới thiệu

44 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Code: inference



Image Segmentation

Mục lục

Giới thiệu

45 Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

```
model.eval()
with torch.no_grad():
    logits = model(images)
    pred = logits.argmax(dim=1)
```

Ghi chú

Nếu cần xác suất, hãy áp dụng softmax lên logits.

Hàm mất mát

Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

46 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Vì sao segmentation không chỉ là “một softmax CE”?



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

47 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Mất cân bằng lớp và mơ hồ ở biên làm pixel CE đơn giản hoạt động chưa đủ tốt.
- Các overlap metrics như Dice/IoU thường gần hơn với mục tiêu triển khai.

Pixel cross-entropy



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

48 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Với logits $\mathbf{Z}$ và target dạng one-hot hoặc class index:

$$\mathcal{L}_{CE} = -\frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \log p_{i, y_i}.$$

- Baseline ổn định; hỗ trợ `ignore_index`.
- Với binary head: thường dùng `BCEWithLogitsLoss` theo pixel hoặc channel.

Dice coefficient (metric)



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

49 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Dice theo từng lớp:

$$\text{Dice}_k = \frac{2 \sum_i p_{i,k} g_{i,k} + \epsilon}{\sum_i p_{i,k} + \sum_i g_{i,k} + \epsilon}.$$

Metric vs loss

Dice **metric** so sánh dự đoán với mask; Dice **loss** thường là $1 - \text{Dice}$.

IoU / Jaccard (metric)



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

50 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

$$\text{IoU}_k = \frac{TP_k}{TP_k + FP_k + FN_k}.$$

Metric vs loss

IoU **loss** thường dùng $1 - \text{IoU}$ hoặc các surrogate mềm như Lovasz.

Dice loss và IoU-style losses



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

51 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Dice loss: $\mathcal{L}_{Dice} = 1 - \text{Dice}$; mạnh với foreground nhỏ nhưng có thể kém ổn định lúc đầu.
- IoU loss: $1 - \text{IoU}$ hoặc soft surrogate; thường kết hợp với CE để ổn định.

Ví dụ tính toán với mask nhỏ: thiết lập



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

52 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Ground truth G

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Prediction $\hat{Y}$

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- Binary case: 1 = foreground, 0 = background.
- Mẹo khi làm bài: đếm **TP**, **FP**, **FN** trước, rồi thay vào công thức Dice/IoU.

Ví dụ: tính Dice và IoU

- Từ hai mask: $TP = 2$ (ô (1, 1), (2, 1)), $FP = 1$ (ô (2, 2)), $FN = 1$ (ô (1, 2)).

$$\text{Dice} = \frac{2TP}{2TP + FP + FN} = \frac{2 \cdot 2}{2 \cdot 2 + 1 + 1} = \frac{4}{6} = 0.667$$

$$\text{IoU} = \frac{TP}{TP + FP + FN} = \frac{2}{2 + 1 + 1} = \frac{2}{4} = 0.5$$

Quan hệ nhanh

Với binary sets, $\text{Dice} = \frac{2\text{IoU}}{1+\text{IoU}}$.

Ví dụ: DiceLoss và IoULoss



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

54 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

$$\mathcal{L}_{Dice} = 1 - \text{Dice} = 1 - 0.667 = 0.333$$

$$\mathcal{L}_{IoU} = 1 - \text{IoU} = 1 - 0.5 = 0.5$$

- Overlap tốt hơn $\Rightarrow$ Dice/IoU cao hơn $\Rightarrow$ DiceLoss/IoULoss thấp hơn.
- Khi huấn luyện, dùng phiên bản **soft** từ xác suất; khi làm bài, thường tính từ hard masks.

Hybrid: CE + Dice

$$\mathcal{L} = \lambda_{ce}\mathcal{L}_{CE} + \lambda_{dice}\mathcal{L}_{Dice}.$$

- CE giúp tối ưu ổn định; Dice kéo mô hình về chất lượng overlap.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

55 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Code: CE + Dice loss



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

56 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

```
ce = nn.CrossEntropyLoss(ignore_index=255)
def ce_dice_loss(logits, target, eps=1e-6, lam=0.5):
    # logits: (B,K,H,W), target: (B,H,W)
    loss_ce = ce(logits, target)
    valid = (target != 255)
    t = target.clone()
    t[~valid] = 0 # safe index for one_hot
    one_hot = F.one_hot(t, num_classes=logits.size(1)).permute(0,3,1,2).float()
    probs = torch.softmax(logits, dim=1)
    valid = valid.unsqueeze(1)
    inter = (probs * one_hot * valid).sum((0,2,3))
    denom = ((probs + one_hot) * valid).sum((0,2,3))
    loss_dice = 1 - ((2*inter + eps) / (denom + eps)).mean()
    return loss_ce + lam * loss_dice
```

Weighted CE cho mất cân bằng lớp

- Class weights w_k tăng penalty cho các lớp hiếm.
- Thường lấy từ inverse frequency, có smoothing hoặc clipping để tránh quá cực đoan.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

57 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Focal và Tversky (ngắn gọn)

- **Focal**: nhấn mạnh các pixel khó; hữu ích khi background áp đảo.
- **Tversky**: cho phép cân bằng FP/FN bất đối xứng, ví dụ bỏ sót lesion nguy hiểm hơn báo động giả.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

58 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Chọn loss trong thực hành

- Bắt đầu: CE + ignore index + kiểm tra augmentation nghiêm túc.
- Thêm Dice/IoU term nếu overlap metrics bị chững.
- Tune λ theo validation, không chỉ nhìn training loss.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

59 Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

Tổng kết

Độ đo và đánh giá

Vì sao evaluation protocol quan trọng?



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

61 Độ đo

Dữ liệu

Nâng cao

Tổng kết

- Metrics định nghĩa thế nào là “tốt hơn” đối với rủi ro triển khai.
- **Engineering**: cùng H,W cho logits và mask sau resize; dùng `ignore_index` nhất quán trong loss và metrics; quyết định rõ void có bị loại khỏi mIoU hay không.
- Sai khác resize/crop có thể làm IoU/Dice sai âm thầm.

IoU và mIoU



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

62 Độ đo

Dữ liệu

Nâng cao

Tổng kết

$$\text{IoU}_k = \frac{TP_k}{TP_k + FP_k + FN_k}, \quad \text{mIoU} = \frac{1}{K} \sum_{k=1}^K \text{IoU}_k.$$

- mIoU xem các lớp ngang nhau theo kiểu macro average.

$$\text{Dice}_k = \frac{2TP_k}{2TP_k + FP_k + FN_k}.$$

- Dice rất phổ biến trong báo cáo medical imaging, thường đi kèm mIoU.

HD95 (Hausdorff 95%)

- Đo sai khác biên bằng khoảng cách giữa các contour.
- Dùng **95th percentile** để giảm ảnh hưởng của outlier.

Khi nào dùng

Các bài toán nhạy với biên: surgical margins, cấu trúc mảnh, mạch máu.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

64 Độ đo

Dữ liệu

Nâng cao

Tổng kết

Pixel accuracy và góc nhìn theo từng lớp



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

65 Độ đo

Dữ liệu

Nâng cao

Tổng kết

$$\text{PixAcc} = \frac{\sum_k TP_k}{\sum_k (TP_k + FN_k)}.$$

- Dễ gây hiểu lầm khi mất cân bằng lớp vì background chiếm đa số.
- Luôn kiểm tra **per-class IoU/Dice**, không chỉ nhìn aggregate metrics.

Cách báo cáo và chọn metric

- Báo cáo split protocol, danh sách lớp, ignore rules, và tiêu chí chọn checkpoint.
- Chọn metric theo chi phí lỗi: biên, vùng, hay instance.

Gợi ý nhanh

Mất cân bằng: mIoU/Dice; biên: HD95 hoặc boundary-F; instance: AP/PQ family.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

66 Độ đo

Dữ liệu

Nâng cao

Tổng kết

Data pipeline và loader

Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

67 Dữ liệu

Nâng cao

Tổng kết

Cấu trúc thư mục điển hình

- images/ và masks/ với tên file tương ứng.
- Hoặc dùng CSV/JSON ánh xạ đường dẫn một cách tường minh.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

68 Dữ liệu

Nâng cao

Tổng kết

Ghép cặp image + mask

- Với semantic segmentation: thường một mask cho mỗi ảnh.
- Kiểm tra shape khớp nhau và hệ tọa độ nhất quán sau resize.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

69 Dữ liệu

Nâng cao

Tổng kết

Mask encoding



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

70 Dữ liệu

Nâng cao

Tổng kết

- **Index map:** mỗi pixel là class id dạng số nguyên, phổ biến với CE.
- **One-hot:** dùng khi cần cho một số loss, nhưng tốn bộ nhớ hơn.

Tensor shapes sau khi load



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

71 Dữ liệu

Nâng cao

Tổng kết

- Image tensor: $(3, H, W)$ hoặc batch $(B, 3, H, W)$.
- Mask: (H, W) dạng long cho CE; kích thước không gian phải khớp logits.

Quy tắc augmentation

- Biến đổi hình học: áp dụng cùng một transform cho image và mask.
- Interpolation cho mask: chỉ dùng **nearest** với label maps.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

72 Dữ liệu

Nâng cao

Tổng kết

Split discipline và leakage



- Chia theo patient/video/scene/source khi phù hợp.
- Không đặt các bản augmented của cùng một mẫu vào các split khác nhau.
- Tránh near-duplicate frames giữa train/val/test, đặc biệt với video datasets.

Takeaway

Leakage có thể tạo mIoU/Dice cao giả và làm hệ thống rất yếu khi triển khai thật.

Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

73 Dữ liệu

Nâng cao

Tổng kết

Visual sanity checks

- Kiểm tra overlay image + mask trước khi huấn luyện.
- Kiểm tra color map và ánh xạ class-id.
- Kiểm tra mask interpolation là nearest-neighbor.
- Kiểm tra ignore label trong cả loss và metrics.
- Kiểm tra một mini-batch sau augmentation.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

74 Dữ liệu

Nâng cao

Tổng kết

PyTorch Dataset pattern



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

75 Dữ liệu

Nâng cao

Tổng kết

```
class SegDataset(torch.utils.data.Dataset):  
    def __getitem__(self, idx):  
        img = load_image(self.paths[idx])  
        msk = load_mask(self.mask_paths[idx])  
        img, msk = self.transforms(img, msk)  
        return img, msk.long()
```

DataLoader



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

76 Dữ liệu

Nâng cao

Tổng kết

```
loader = torch.utils.data.DataLoader(  
    ds, batch_size=8, shuffle=True,  
    num_workers=4, pin_memory=True,  
)
```

- Dùng `collate_fn` nếu tensor có kích thước biến đổi cần padding.

Ignore index, mất cân bằng, checklist lỗi



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

77 Dữ liệu

Nâng cao

Tổng kết

- `ignore_index` cho void pixels phải khớp với encoding của dataset.
- Vẽ histogram tần suất lớp trước khi chọn loss weights.
- Checklist: mask interpolation sai, lệch nhãn off-by-one, train/val leakage, resize mismatch.



Chủ đề nâng cao

Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

78 Nâng cao

Tổng kết

DeepLab + ASPP



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

79 Nâng cao

Tổng kết

- Atrous conv mở rộng receptive field mà không làm giảm độ phân giải quá sớm.
- ASPP kết hợp context đa tỉ lệ ở một output stride cố định.

Khi nào dùng

Cần context ngữ nghĩa mạnh cho scene parsing với chi phí tính toán vừa phải.

- Hierarchical transformer encoder + lightweight MLP decoder.

Khi nào dùng

Cần độ chính xác tốt với decoder hiệu quả cho semantic segmentation.

- Mask classification với transformer decoder queries.

Khi nào dùng

Muốn thống nhất semantic/instance/panoptic segmentation trong một framework.

SAM (Segment Anything)



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

82 Nâng cao

Tổng kết

- Promptable foundation model với points/boxes/masks.

Khi nào dùng

Interactive annotation, rapid prototyping; có thể cần adaptation khi domain shift.

Weak supervision



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

83 Nâng cao

Tổng kết

- Nhãn có thể đến từ boxes, scribbles, image tags; thường kết hợp pseudo-label loops.

Trade-off

Giảm chi phí gán nhãn nhưng tăng độ phức tạp thuật toán và yêu cầu xử lý nhiều.

Semi-supervised segmentation



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

84 Nâng cao

Tổng kết

- Consistency regularization + confidence filtering trên ảnh chưa gán nhãn.

Khi nào dùng

Có nhiều ảnh nhưng ít nhãn pixel-level.

Hướng foundation model



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

85 Nâng cao

Tổng kết

- Dùng shared backbones + adapters cho các downstream dense tasks.

Takeaway

Xem mô hình lớn như teacher hoặc feature provider; không phải lúc nào cũng là final runtime model.

Gợi ý tài nguyên

- Surveys và benchmarks: tìm “cityscapes benchmark”, “COCO panoptic”.
- Implementations: `segmentation_models_pytorch`, `mmsegmentation`, `Detectron2`.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

86 Nâng cao

Tổng kết

References (UNet taxonomy)



- 1 O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.
- 2 L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking Atrous Convolution for Semantic Image Segmentation," arXiv:1706.05587, 2017.
- 3 J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proc. CVPR*, 2018, pp. 7132–7141.
- 4 S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in *Proc. ECCV*, 2018, pp. 3–19.
- 5 A. Vaswani *et al.*, "Attention Is All You Need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.
- 6 T.-Y. Lin *et al.*, "Feature Pyramid Networks for Object Detection," in *Proc. CVPR*, 2017, pp. 2117–2125.
- 7 M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," in *Proc. CVPR*, 2020, pp. 10781–10790.
- 8 E. Xie *et al.*, "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *Proc. NeurIPS*, 2021.
- 9 B. Cheng *et al.*, "Masked-attention Mask Transformer for Universal Image Segmentation," in *Proc. CVPR*, 2022, pp. 1290–1299.
- 10 A. Kirillov *et al.*, "Segment Anything," in *Proc. ICCV*, 2023, pp. 4015–4026.
- 11 Z. Zhang, Q. Liu, and Y. Wang, "Road Extraction by Deep Residual U-Net," *IEEE Geoscience and Remote Sensing Letters*, vol. 15, no. 5, pp. 749–753, 2018.

Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

87 Nâng cao

Tổng kết

Công thức thực hành cho một project mới

- ❶ Chốt task type: semantic/instance/panoptic và metric chính.
- ❷ Xây dựng dataset contract + visualization sanity checks.
- ❸ Bắt đầu với UNet + CE; sau đó thêm overlap loss nếu cần.
- ❹ Debug bằng per-class metrics và qualitative grids.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

88 Tổng kết

Các lỗi thường gặp

- Bilinear resize mask; ignore labels không nhất quán.
- Tune trên test set; metric và loss không ăn khớp.
- Chỉ báo cáo pixel accuracy khi dữ liệu mất cân bằng.



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

89 Tổng kết

Sinh viên cần nhớ gì?



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

90 Tổng kết

- Dense outputs cần **spatial reconstruction**: decoder/skips.
- Loss và metric phải khớp với label encoding và chi phí lỗi của bài toán.
- Chất lượng data pipeline chi phối rất nhiều lỗi kiểu “model không học”.

Tổng kết



Image Segmentation

Mục lục

Giới thiệu

Kiến trúc Unet

Hàm mất mát

Độ đo

Dữ liệu

Nâng cao

91 Tổng kết

- Segmentation = structured prediction: tối ưu cục bộ, đánh giá toàn cục.
- UNet là mental model CNN cốt lõi cho dense decoding.
- Engineering: loaders, losses, metrics, và protocols là kỹ năng hạng nhất.